API Versioning, Error Shape, Rate Limiting & Docs

11 min read Updated 24 Jul 2026 Topic 42 of 70

Why it matters

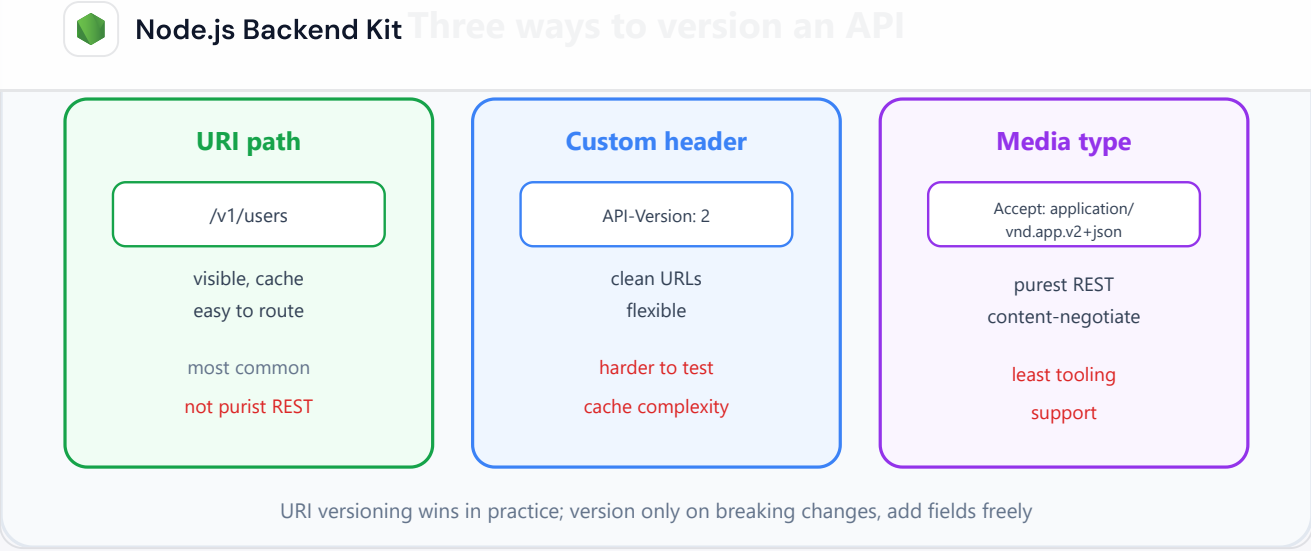
The moment your API has a single external consumer you cannot control, you have lost the freedom to change it freely. Rename a field, tighten a validation rule, or change a response shape, and somewhere a mobile app in a user's pocket — running a build from six months ago — breaks. **Versioning** is the discipline that lets a server evolve while old clients keep working. Interviewers ask about it because it reveals whether you have maintained a real API over time or only built greenfield toys.

The other three topics here — error shape, rate limiting, and documentation — are the difference between an API developers *enjoy* using and one they curse. A **consistent error shape** means every client can write one error handler instead of guessing which of five formats today's endpoint uses. **Rate limiting** protects you from abuse, runaway clients, and cost blowups — and is a near-guaranteed system-design question ("how would you stop one user from hammering this endpoint?"). And **documentation**, specifically OpenAPI, is what makes an API discoverable and testable without reading your source; "have you used OpenAPI/Swagger?" is a common screening question because it signals you've worked on APIs meant for others.

Together these four turn a working endpoint into a *product*: stable, safe, self-describing, and pleasant to integrate against.

The concept, simply

Versioning lets multiple contract versions coexist so a breaking change ships as "v2" without forcing every client to upgrade at once. There are three mainstream strategies.



Three ways to version. URI-path versioning wins in practice for its visibility and simple routing.

- **URI path:** `/v1/users` vs `/v2/users`. Visible, cache-friendly, trivial to route. The most common choice.
- **Custom header:** `API-Version: 2`. Keeps URLs clean but is invisible and harder to test in a browser.
- **Media type (content negotiation):** `Accept: application/vnd.myapp.v2+json`. The most "RESTful," the least tooled and understood.

A **breaking change** is one that could break an existing client: removing/renaming a field, changing a type, adding a required input, or changing status-code semantics. *Additive* changes — adding an optional field or a new endpoint — are backward-compatible and do *not* need a new version. The golden rule: version only on breaking changes; add fields freely.

A **consistent error shape** is a single JSON structure every error returns — e.g. `{ error: { code, message, details } }` — so clients parse errors uniformly. **Rate limiting** caps how many requests a client may make in a window, returning **429 Too Many Requests** when exceeded. **OpenAPI** (formerly Swagger) is a machine-readable YAML/JSON spec describing your endpoints, from which docs, client SDKs, and test tools are generated.

How it works (deep)

Versioning strategy trade-offs. URI-path versioning wins in practice despite purists objecting that `/v1/` means the "same" resource has two URIs (technically un-RESTful). It wins because it is *visible* (you can see the version in logs and browsers), *routable*

tooling: you cannot paste a versioned URL into a browser, CDNs need to vary on the header, and debugging is harder. A senior answer names URI-path as the pragmatic default while acknowledging the REST-purity critique.

The deeper skill is *minimizing* versions. Each live version is maintenance forever. So: design responses to be extensible from day one (return objects, not bare arrays, so you can add fields), treat new fields as additive, and only cut a new version for genuinely breaking changes. When you do, run old and new side by side, announce a *deprecation* timeline (often via a `Deprecation` / `Sunset` header), and give clients months to migrate.

Error shape. The failure mode to avoid is every endpoint inventing its own error format — one returns `{ error: "..." }`, another `{ message: "..." }`, a third a bare string. Standardize on one envelope with a *machine-readable* `code` (so clients branch on `code === 'EMAIL_TAKEN'` rather than string-matching human text), a human `message`, and optional `details` for field-level validation errors. Centralize it in a single Express **error-handling middleware** (the four-argument `(err, req, res, next)` signature) so no handler formats errors ad hoc, and so you can map error *types* to status codes in one place. Critically, never leak stack traces or raw DB errors to clients — log those server-side, return a clean generic message for 5xx.

Rate limiting algorithms. The common ones: *fixed window* (count requests per clock minute — simple but allows bursts at window edges, up to 2× the limit across a boundary); *sliding window* (smooths that edge burst); *token bucket* (a bucket refills at a steady rate and each request spends a token — allows controlled bursts, the most popular); and *leaky bucket* (processes at a fixed rate, smoothing output). For a single Node process an in-memory counter works, but the moment you run multiple instances you need a *shared store* — typically Redis — because each instance otherwise keeps its own count and the real limit becomes N× your intended value. Always return **429** plus `Retry-After` and the `RateLimit-*` headers so well-behaved clients back off.

OpenAPI. The spec is a contract written in YAML/JSON describing paths, methods, request/response schemas, and auth. Its value is that it is machine-readable: Swagger UI renders interactive docs from it, code generators produce typed client SDKs and server stubs, and contract-testing tools verify your implementation matches the spec. You can

documentation cannot silently drift from the real API.

Code examples

URI-path versioning by mounting separate routers — v1 and v2 evolve independently.

JAVASCRIPT Copy

```
const express = require('express');
const app = express();

const v1 = require('./routes/v1');
const v2 = require('./routes/v2');

app.use('/v1', v1); // legacy clients keep working
app.use('/v2', v2); // new shape, breaking changes live here

// Signal deprecation so clients know to migrate
app.use('/v1', (req, res, next) => {
  res.set('Deprecation', 'true');
  res.set('Sunset', 'Wed, 31 Dec 2026 23:59:59 GMT');
  next();
});
```

A single, consistent error shape via centralized error-handling middleware — note the four arguments and the 4xx/5xx split:

JAVASCRIPT Copy

```
// Throw typed errors anywhere in the app
class ApiError extends Error {
  constructor(status, code, message, details) {
    super(message);
    this.status = status; this.code = code; this.details = details;
  }
}

// One place formats EVERY error (must have 4 args to be an error handler)
function errorHandler(err, req, res, next) {
```

```
    ? { error: { code: 'INTERNAL', message: 'Something went wrong' } }
    : { error: { code: err.code, message: err.message, details: err.detail

  if (status ≥ 500) console.error(err); // log real cause server-side
  res.status(status).json(body);
}

app.use(errorHandler); // mounted LAST, after all routes
```

Rate limiting with a Redis-backed store so the limit is correct across many instances:

JAVASCRIPT Copy

```
const rateLimit = require('express-rate-limit');
const { RedisStore } = require('rate-limit-redis');
const redis = require('./redisClient');

const limiter = rateLimit({
  windowMs: 60 * 1000,          // 1-minute window
  max: 100,                     // 100 requests per key per window
  standardHeaders: true,        // RateLimit-* headers
  legacyHeaders: false,
  store: new RedisStore({ sendCommand: (...args) ⇒ redis.call(...args) })
  keyGenerator: (req) ⇒ req.user?.id || req.ip, // per user, fallback per
  handler: (req, res) ⇒ {
    res.set('Retry-After', '60');
    res.status(429).json({ error: { code: 'RATE_LIMITED', message: 'Too ma
  },
});

app.use('/api', limiter);
```

A minimal OpenAPI fragment documenting one endpoint, plus serving interactive docs with Swagger UI:

JAVASCRIPT Copy

```
//   /v2/users/{id}:
//     get:
//       summary: Get a user by id
//       parameters:
//         - name: id
//           in: path
//           required: true
//           schema: { type: string }
//       responses:
//         '200': { description: OK }
//         '404': { description: Not found }

const swaggerUi = require('swagger-ui-express');
const YAML = require('yamljs');
const spec = YAML.load('./openapi.yaml');

app.use('/docs', swaggerUi.serve, swaggerUi.setup(spec)); // interactive d
```

Common mistakes & gotchas

- **Versioning for additive changes:** adding an optional field doesn't need v2. Over-versioning multiplies maintenance for nothing.
- **Never deprecating old versions:** without a sunset date, v1 lives forever. Announce timelines and use `Deprecation` / `Sunset` headers.
- **Inconsistent error shapes:** different formats per endpoint force clients to write N error handlers. Standardize one envelope.
- **Leaking stack traces / DB errors** to clients on 5xx: an information-disclosure risk. Log server-side, return a generic message.
- **Error middleware with three args:** Express only treats a middleware as an error handler if it has *four* parameters. Drop `next` and errors silently bypass it.
- **In-memory rate limiting across multiple instances:** each process counts separately, so the real limit becomes N× intended. Use Redis.
- **Rate limiting by IP only:** corporate NAT/mobile networks share IPs; prefer per-user (with IP fallback) where you have identity.

- **Docs that drift from code:** hand-maintained docs rot. Generate from or contract-test against the OpenAPI spec.
- **Returning a bare array** as the top-level response: you can't later add pagination/metadata without breaking clients. Wrap in an object.

Best practices

- Default to **URI-path versioning** (`/v1`, `/v2`); version only on **breaking** changes, add fields freely.
- Design responses as **extensible objects** (never bare arrays) so additive changes stay backward-compatible.
- Publish a **deprecation policy**: overlap versions, send `Deprecation`/`Sunset` headers, give clients months.
- Standardize **one error envelope** with a machine-readable `code`, human `message`, and optional `details`; format it in one central error-handling middleware.
- On 5xx, **log the real cause, return a generic message** — never leak internals.
- **Rate-limit** with a shared store (Redis) across instances, key by user then IP, and return **429** + `Retry-After` + `RateLimit-*` headers.
- Maintain an **OpenAPI** spec as the single source of truth; serve Swagger UI and generate clients/tests from it so docs can't drift.

⭐ Interview lens

Q: Which API versioning strategy would you choose and why?
URI-path (`/v1`) in practice: visible in logs/browsers, trivial to route (mount separate routers), and cacheable per-URL. Purists note it's technically un-RESTful (same resource, two URIs) and prefer media-type versioning — but that's poorly tooled. Naming the trade-off, not just the choice, is what scores.

Q: What counts as a breaking change requiring a new version?
Removing/renaming a field, changing a field's type, adding a required request param, or changing status-code semantics. Adding an *optional* field or a new

endpoint is additive and backward-compatible — no new version needed. Version only on breaking changes.

Q: How would you rate-limit an endpoint across a horizontally-scaled service?

A shared counter in Redis (token bucket or sliding window), keyed by user id (IP fallback), returning 429 with `Retry-After`. The trap is in-memory counters: with N instances each keeps its own count, so the effective limit becomes N× the intended value.

Q: Why must your Express error handler take four arguments?

Express identifies error-handling middleware by arity — only a function with exactly `(err, req, res, next)` is treated as one. Write three params and Express treats it as normal middleware, so thrown errors skip it and fall through to the default handler.

Q: What does a good error response look like?

A single consistent envelope: a machine-readable `code` (clients branch on it, not on message text), a human `message`, and optional `details` for validation. Formatted in one central middleware; on 5xx it hides internals and logs the real cause server-side. Consistency lets clients write one handler.

Q: What is OpenAPI and why use it?

A machine-readable spec (YAML/JSON) describing paths, schemas, and auth. From it you generate interactive docs (Swagger UI), typed client SDKs, and contract tests — one source of truth so docs can't drift from the implementation. Saying "Swagger" and OpenAPI are the same lineage (Swagger was renamed OpenAPI) shows familiarity.

Fixed window counts per clock interval; it's simple but permits a burst of up to 2× the limit straddling a boundary. Token bucket refills tokens at a steady rate and each request spends one, allowing controlled bursts up to the bucket size while enforcing an average rate — smoother, and the common production choice.

Cheat-card

- **Version on breaking changes only**; add optional fields freely. Default: URI-path `/v1`, `/v2`.
- **Breaking** = remove/rename field, change type, new required input, changed status semantics.
- Deprecate with overlap + `Deprecation` / `Sunset` headers and a migration window.
- **One error envelope:** `{ error: { code, message, details } }`; central 4-arg middleware; hide internals on 5xx.
- Express error handler needs **4 args** `(err, req, res, next)` or it's skipped.
- **Rate limit:** Redis-shared across instances, key by user/IP, return **429** + `Retry-After`. Token bucket = smooth bursts.
- **OpenAPI/Swagger:** one spec → docs + SDKs + contract tests; keeps docs from drifting.
- Return **objects, not bare arrays**, so responses stay extensible.

QUICK CHECK — no marks, just for you.

1. Which change is a breaking change that warrants a new API version?

- ◯ Adding a new optional field to a response
- ◯ Removing or renaming a response field, or adding a required request parameter
- ◯ Adding a brand-new endpoint alongside the existing ones

Fixing a typo in the human-readable error message text

Check answer

2. Why must an Express error-handling middleware take exactly four arguments?

- ◯ The fourth argument carries the response body to send
- ◯ Four arguments make the handler run first instead of last
- ◯ Express identifies error handlers by arity; only a function with (err, req, res, next) is treated as one
- ◯ It is a style convention with no effect on behaviour

Check answer

3. Why does in-memory rate limiting break across a horizontally-scaled service?

- ◯ In-memory counters reset on every request, so no limit is ever enforced
- ◯ The counters are shared automatically, but they overflow past 100
- ◯ Rate limiting can only ever be done at the database layer
- ◯ Each instance keeps its own counter, so with N instances the effective limit becomes N times the intended value

Check answer

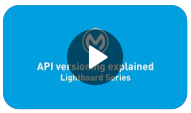
Node.js Backend Kit

LEARN MORE

ARTICLES & DOCS

- Web API Design Best Practices (API Versioning) - Azure Architecture Center
  learn.microsoft.com
- Microsoft Azure REST API Guidelines - API Versioning
  github.com

VIDEOS



API Versioning Explained | Lightboard Series
MuleSoft Videos

Hand-picked references for this lesson.